ENTROPY AND INFORMATION THEORY IN NLP

Python NLP Laboratory — Step-by-step execution and explanation

Objective: Calculate the entropy and cross-entropy of a text corpus using word-probability distributions.

1. What is Entropy?

Entropy measures the amount of uncertainty or unpredictability in a probability distribution. In NLP, it can describe how uncertain the next word is when words are modelled using their probabilities.

Example A: If a word is always the same, such as “AI AI AI AI”, the word distribution has no uncertainty and its entropy is zero.
Example B: If several words have equal probabilities, it is harder to predict which word will occur, so entropy is higher.

Entropy formula: $H(P) = -\Sigma P(w) \log_2 P(w)$

P(w) is the probability of a word. The logarithm is base 2, so entropy is measured in bits per word.

2. What is Cross-Entropy?

Cross-entropy measures how well one probability distribution represents or predicts another. In this experiment, P is the actual word distribution and Q is the model's estimated word distribution. If Q assigns high probabilities to words that are common in P, cross-entropy tends to be lower.

Cross-entropy formula: $H(P,Q) = -\Sigma P(w) \log_2 Q(w)$

Important distinction: Entropy measures uncertainty in the actual distribution. Cross-entropy measures the cost of representing the actual distribution using the model distribution.

3. Experiment workflow

The program creates two corpora, tokenizes their sentences, counts words, calculates word probabilities, computes entropy and cross-entropy, and displays the results.

4. Complete Python program

Create a file named **entropy_cross_entropy.py** and enter this code:

```
from collections import Counter
import math

# STEP 1: Create two text corpora

# Actual text corpus
corpus_P = [
    "data science is useful",
    "data science is interesting",
    "machine learning is useful"
]

# Predicted/model text corpus
corpus_Q = [
    "data science is useful",
    "machine learning is useful",
    "data science is useful",
    "machine learning is interesting"
]


# STEP 2: Tokenization and word counting

words_P = []
for sentence in corpus_P:
```

```
    words_P.extend(sentence.lower().split())

words_Q = []
for sentence in corpus_Q:
    words_Q.extend(sentence.lower().split())

count_P = Counter(words_P)
count_Q = Counter(words_Q)

total_P = len(words_P)
total_Q = len(words_Q)


# STEP 3: Calculate word probabilities

vocabulary = sorted(set(words_P) | set(words_Q))

P = {}
Q = {}

for word in vocabulary:
    P[word] = count_P[word] / total_P
    Q[word] = count_Q[word] / total_Q


# STEP 4: Display word probabilities

print("WORD PROBABILITY DISTRIBUTIONS")
print("\nWord\t\tP (Actual)\tQ (Model)")

for word in vocabulary:
    print(word, "\t\t",
          round(P[word], 4), "\t\t",
          round(Q[word], 4))


# STEP 5: Calculate Entropy

entropy = 0
for word in vocabulary:
    if P[word] > 0:
        entropy -= P[word] * math.log2(P[word])


# STEP 6: Calculate Cross-Entropy

cross_entropy = 0

for word in vocabulary:
    if P[word] > 0:
        if Q[word] == 0:
            cross_entropy = float("inf")
            break

        cross_entropy -= P[word] * math.log2(Q[word])


# STEP 7: Display results

print("\nENTROPY:", round(entropy, 4), "bits/word")
print("CROSS-ENTROPY:",
      round(cross_entropy, 4), "bits/word")
```

5. Explanation of the program

Step 1 — Create two text corpora

Corpus P is the actual text corpus. Corpus Q is used as the model's estimated word distribution. These are small sample corpora for demonstrating the calculations; Q is not a trained neural language model.

Step 2 — Tokenization and word counting

```
words_P = []
for sentence in corpus_P:
    words_P.extend(sentence.lower().split())
```

```
count_P = Counter(words_P)
```

lower() converts the text to lowercase. **split()** separates a sentence into words. **extend()** adds all words from a sentence to the list. **Counter()** counts how often each word occurs.

The actual corpus P contains 12 word tokens:

Word	Frequency in P
data	2
science	2
is	3
useful	2
interesting	1
machine	1
learning	1
Total	12

Step 3 — Calculate word probabilities

The probability of a word is its frequency divided by the total number of word tokens. For example, in P: P(data) = 2/12 = 0.1667 and P(is) = 3/12 = 0.25. The program calculates this for every word in the combined vocabulary, separately for P and Q.

Step 4 — Display probability distributions

Word	P (Actual)	Q (Model)
data	0.1667	0.1250
interesting	0.0833	0.0625
is	0.2500	0.2500
learning	0.0833	0.1250
machine	0.0833	0.1250
science	0.1667	0.1250
useful	0.1667	0.1875

P represents the actual word-probability distribution, while Q represents the model's estimated distribution. Each column sums to 1 (allowing for rounding).

Step 5 — Calculate entropy

```
entropy = 0
for word in vocabulary:
    if P[word] > 0:
        entropy -= P[word] * math.log2(P[word])
```

The program starts entropy at zero. For each word, it multiplies its actual probability by the base-2 logarithm of that probability, then subtracts the result from the running total. For data, the contribution is –0.1667 × log■(0.1667) ≈ 0.4308 bits.

Entropy of P = 2.6887 bits per word.

Step 6 — Calculate cross-entropy

```
cross_entropy = 0
for word in vocabulary:
    if P[word] > 0:
        if Q[word] == 0:
            cross_entropy = float("inf")
```

```
            break
        cross_entropy -= P[word] * math.log2(Q[word])
```

This calculation uses P[word] for the actual probability and Q[word] for the model probability. For data, the contribution is –0.1667 × log■(0.125) = 0.5 bits. If an actual word has positive P but Q is zero, cross-entropy is infinite; the program checks for this case.

Cross-entropy H(P,Q) = 2.7358 bits per word.

6. Expected output

```
WORD PROBABILITY DISTRIBUTIONS

Word          P (Actual)    Q (Model)
data          0.1667        0.125
interesting   0.0833        0.0625
is            0.25          0.25
learning      0.0833        0.125
machine       0.0833        0.125
science       0.1667        0.125
useful        0.1667        0.1875

ENTROPY: 2.6887 bits/word
CROSS-ENTROPY: 2.7358 bits/word
```

7. Interpreting the results

Entropy of the actual distribution is 2.6887 bits per word. Cross-entropy using the model distribution is 2.7358 bits per word. The cross-entropy is slightly higher because Q does not exactly match P. A perfect match Q = P would make cross-entropy equal to entropy.

8. Entropy versus cross-entropy

Entropy	Cross-Entropy
Measures uncertainty in one distribution.	Measures the cost of representing P using Q.
Uses –Σ P(w) log■ P(w).	Uses –Σ P(w) log■ Q(w).
Uses the actual distribution P.	Uses actual P and model Q.

9. How to execute the program

1. Open IDLE, VS Code, PyCharm or Jupyter Notebook.
2. Create a new Python file named **entropy_cross_entropy.py**.
3. Paste the complete program and save the file.
4. Open Command Prompt or a terminal in the folder where the file is saved.
5. Run the command:

```
python entropy_cross_entropy.py
```

No external libraries are required. The program uses Python's built-in **collections** module and the standard **math** module.

10. Student exercises

Question 1: What does entropy measure?
Answer: Uncertainty in a probability distribution.

Question 2: Which formula represents cross-entropy?
Answer: H(P,Q) = –Σ P(w) log■ Q(w).

Question 3: What happens if Q assigns zero probability to a word that has positive probability in P?
Answer: Cross-entropy becomes infinite. Smoothing is commonly used to avoid zero probabilities.

11. Important technical note

This introductory experiment uses unigram word distributions. It does not calculate sentence-level or sequential language-model cross-entropy. Corpus Q is a frequency-based sample distribution, not a trained predictive language model. In practical NLP, smoothing or other probability-estimation methods are often used so that unseen words do not receive zero probability.

Expected laboratory outcome: Students should be able to calculate word-probability distributions, determine entropy, calculate cross-entropy, and interpret the results in the context of NLP language models.